

---

# Tổng quan tài liệu Project 15

Dự án: Xây dựng agent harness tùy chỉnh và vượt qua một harness đã được thiết lập trên Terminal-Bench

Môn học: 36127 Innovation Lab: Capstone Project

Biên soạn cho: Nhóm Project 15 gồm sáu thành viên

Ngày: 30 tháng 7 năm 2026

## Bối cảnh dự án

Dự án 15 yêu cầu nhóm xây dựng agent harness tùy chỉnh và kiểm tra xem liệu nó có thể hoạt động tốt hơn harness đã thiết lập trên Terminal-Bench trong khi giữ nguyên mô hình ngôn ngữ cơ bản hay không. Bản tóm tắt dự án thu hẹp phạm vi dự định:

- sử dụng Terminal-Bench 2.1 và 89 tác vụ đầu cuối của nó;
- so sánh với hai harness đã được thiết lập;
- giữ mô hình không đổi sao cho những khác biệt về hiệu suất có thể được quy cho thiết kế harness một cách đáng tin cậy hơn;
- phát triển trên một tập hợp con cố định gồm 20 nhiệm vụ;
- thay đổi một đòn bẩy thiết kế tại một thời điểm;
- chỉ chạy benchmark đầy đủ sau khi thiết kế harness tùy chỉnh bị đóng băng;
- ghi lại độ chính xác, mã thông báo, chi phí và thay đổi thiết kế; và
- gửi harness kết quả lên bảng xếp hạng công khai nếu tài nguyên và giao thức cuối cùng cho phép.

Khung này làm cho Capstone trở thành một dự án kỹ thuật phần mềm thử nghiệm có kiểm soát. Sản phẩm chính không chỉ đơn thuần là agent để hoàn thành các nhiệm vụ cuối cùng. Nhóm phải đưa ra bằng chứng có thể bảo vệ được về những quyết định harness nào giúp ích, quyết định nào không, và trong những điều kiện về chi phí và độ tin cậy.

Tài liệu được chọn để đánh giá này giải quyết năm câu hỏi bổ sung:

- 1 Terminal-Bench đo lường điều gì và nó bộc lộ những thất bại phổ biến nào của agent?
- 2 Làm thế nào một giao diện được thiết kế cho LLM có thể cải thiện hiệu suất mà không thay đổi trọng lượng mô hình?
- 3 Những thành phần nào xuất hiện trong một nền tảng tác nhân phần mềm chung?
- 4 Khi nào một quy trình làm việc đơn giản, hạn chế có thể hoạt động tốt hơn một quy trình làm việc tự động phức tạp agent?
- 5 Hệ thống agent nên được cấu trúc như thế nào để có khả năng tái tạo, phục hồi và thử nghiệm đáng tin cậy?

## Tóm tắt điều hành

Năm bài báo đưa ra một kết luận nhất quán: mô hình cơ bản chỉ là một phần của hệ thống tác nhân. Hiệu suất cũng phụ thuộc vào giao diện mô hình-môi trường, thông tin được lưu giữ trong ngữ cảnh, phản hồi được cung cấp sau hành động, các bước kiểm tra được thực hiện trước khi hoàn thành và các cơ chế được sử dụng để khôi phục sau lỗi. Terminal-Bench cung cấp 89 nhiệm vụ đầu cuối khó khăn, đã được xác minh kết quả. Phân tích lỗi của nó nêu bật các hành vi liên quan trực tiếp đến harness thiết kế: không tuân theo các thông số kỹ thuật, lặp lại các bước không hiệu quả, dừng sớm, tạo ảo giác về kết quả, không xác minh các yêu cầu cốt lõi và tuyên bố thành công bất chấp bằng chứng mâu thuẫn. Terminal-Bench 2.1 sau đó đã khắc phục sự cố ở 28 trong số 89 tác vụ ban đầu, chứng minh rằng benchmark cấu hình và verifier chất lượng có thể thay đổi đáng kể hiệu suất đo được. SWE-agent đưa ra bằng chứng trực tiếp mạnh mẽ nhất cho thấy thiết kế giao diện quan trọng. Nó giới thiệu agent-computer interface (ACI) và cho thấy rằng các hành động đơn giản, thao tác nhỏ gọn, phản hồi ngắn gọn và các biện pháp bảo vệ có thể cải thiện agent trong khi vẫn giữ mô hình cố định. Việc cắt bỏ nó cho thấy rằng tìm kiếm tóm tắt, chế độ xem tệp bị giới hạn, thao tác chỉnh sửa nhỏ gọn, tìm lỗi mã nguồn và giảm lịch sử quan sát có thể hiệu quả hơn so với việc chỉ hiển thị một lớp vỏ không có cấu trúc.

OpenHands thể hiện chiều rộng của nền tảng tác nhân phần mềm chung: agent logic, công cụ, thực thi hộp cát, lịch sử sự kiện, quản lý trạng thái, trừu tượng hóa mô hình, ứng dụng và benchmark bộ điều hợp. Nó có giá trị như một đường cơ sở và tài liệu tham khảo kiến trúc đã được thiết lập, nhưng bề rộng của nó cũng là một cảnh báo chống lại việc tái tạo một nền tảng sản xuất trong Capstone một học kỳ.

Agentless cho thấy quyền tự chủ phức tạp không phải lúc nào cũng cần thiết. Một quy trình công việc bản địa hóa, sửa chữa và xác nhận theo giai đoạn có thể đạt được kết quả tốt với chi phí tương đối thấp. Lựa chọn ứng viên và thử nghiệm tái tạo là trung tâm của hiệu suất của nó. Đối với Dự án 15, điều này hỗ trợ bắt đầu bằng một vòng lặp tối thiểu, có thể hiểu được và chỉ thêm quyền tự chủ khi các thử nghiệm được kiểm soát chứng minh được lợi ích.

Bài viết OpenHands SDK đóng góp các bài học thiết kế theo định hướng sản xuất: tách logic agent khỏi các ứng dụng, giữ cấu hình không thay đổi, duy trì một trạng thái có thẩm quyền duy nhất, lưu trữ các hành động và quan sát dưới dạng sự kiện, đồng thời tách biệt các lỗi cơ sở hạ tầng khỏi các lỗi mô hình. Những thực hành này đặc biệt quan trọng đối với Capstone trong đó mọi kết quả đều phải có thể tái tạo và giải thích được.

Do đó, harness ban đầu được đề xuất là một Harbor agent bên ngoài tối thiểu với:

- quy trình làm việc prompt cố định và xác minh kế hoạch rõ ràng;
- một giao diện thực thi thiết bị đầu cuối ban đầu;
- bối cảnh giới hạn với một bản tóm tắt trạng thái nhỏ gọn;
- hồ sơ quan sát và hành động có cấu trúc;
- hoàn thành được kiểm soát bằng việc xác minh được quan sát;
- một cơ hội sửa chữa khi phát hiện lỗi;
- cấu hình bất biến cho mỗi lần chạy; và
- số liệu về độ chính xác, mã thông báo, chi phí, thời gian chạy, số lần thử và loại lỗi.

Các thử nghiệm đầu tiên sẽ kiểm tra cấu trúc prompt, cổng xác minh, sửa chữa nhận biết lỗi, quản lý bối cảnh và mức độ chi tiết của công cụ theo thứ tự đó. Việc phối hợp nhiều tác nhân, tinh chỉnh, truy cập web không bị giới hạn và hệ thống bộ nhớ phức tạp vẫn nằm ngoài phạm vi ban đầu.

## Thuật ngữ chính

| Thuật ngữ                      | Định nghĩa sử dụng trong Project 15                                                                                                      |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Model                          | Mô hình ngôn ngữ lớn cơ bản tạo ra lập luận, lệnh và phản hồi. Model phải được giữ cố định trong quá trình so sánh harness có kiểm soát. |
| Agent                          | Model cộng với chính sách hoặc vòng lặp quyết định cách hành động trong một môi trường.                                                  |
| Harness                        | Hệ thống bao quanh, kết nối model với nhiệm vụ, công cụ, thực thi, ngữ cảnh, ghi log, retry và đánh giá.                                 |
| Agent-computer interface (ACI) | Các hành động có sẵn cho agent và các quan sát được môi trường trả về.                                                                   |

| Thuật ngữ              | Định nghĩa sử dụng trong Project 15                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------------|
| Harbor                 | Framework đánh giá dùng để chạy agent trong môi trường tác vụ container hóa và thu thập kết quả trial.           |
| Nhiệm vụ               | Một hướng dẫn, môi trường container, chính sách tài nguyên và verifier xác định một bài toán benchmark.          |
| Verifier               | Các test hoặc phép kiểm tra tự động dùng để đánh giá trạng thái cuối cùng của môi trường.                        |
| Trajectory             | Bản ghi có thứ tự của prompt, hành động, quan sát, kết quả công cụ, thay đổi trạng thái và quá trình hoàn thành. |
| Baseline               | Một harness đã được thiết lập, chạy với cùng model và điều kiện benchmark như harness tùy chỉnh.                 |
| Tập hợp con phát triển | Tập hợp cố định gồm 20 nhiệm vụ, dùng để thiết kế và so sánh các thay đổi của harness.                           |
| Đánh giá cuối cùng     | Harness tùy chỉnh đã được đồng bộ, chạy trên toàn bộ 89 tác vụ benchmark theo giao thức đã thống nhất.           |
| Cắt bỏ                 | Sự so sánh có kiểm soát trong đó một thành phần được loại bỏ hoặc thay đổi để ước tính sự đóng góp của nó.       |
| Reward hacking         | Đạt được điểm verifier mà không thực sự đáp ứng kết quả dự định của nhiệm vụ.                                    |

# Ý nghĩa dự án và phân tích thực tế

## Tình trạng bằng chứng và ranh giới nguồn

Chương này phân tách các yêu cầu và bằng chứng theo xuất xứ:

- Tóm tắt chính thức của Dự án 15: bản mô tả chủ đề bằng văn bản do Tiến sĩ William So của Synogize cung cấp trong danh sách dự án Capstone Mùa xuân 2026.
- Yêu cầu đối với người hướng dẫn ở cấp độ môn học: Canvas thông báo thành lập nhóm và 36127 slide khởi động.
- Hướng dẫn của cố vấn được ghi lại: chưa có cuộc họp cố vấn Dự án 15 hoặc quyết định của khách hàng nào được ghi vào cuộc họp và nhật ký quyết định.
- Phân tích nhóm: các diễn giải, đánh giá rủi ro, lập bản đồ ứng dụng và đề xuất bên dưới. Đây là những đề xuất để thảo luận, không phải hướng dẫn của người cố vấn hoặc khách hàng.
- Bằng chứng bên ngoài hiện tại: các bản phát hành, tài liệu nghiên cứu và báo cáo kỹ thuật benchmark gần đây được sử dụng để đánh giá xem liệu thiết kế khai thác tác nhân có phải là vấn đề kịp thời hay không.

Sự khác biệt này rất quan trọng đối với tính liên chính trong học thuật và quản trị dự án. Nhóm không nên trình bày giả định lập kế hoạch như một yêu cầu của khách hàng. Sau mỗi cuộc họp với người cố vấn hoặc khách hàng, các quyết định đã được xác nhận phải được thêm vào nhật ký cuộc họp và phản ánh trong bản đánh giá này nếu có liên quan.

## Cơ sở lý luận chính thức của Dự án 15

Bản tóm tắt chính thức mô tả Terminal-Bench 2.1 dưới dạng benchmark công khai chứa 89 nhiệm vụ cuối cùng và bảng xếp hạng gồm harness-model cập. Quan sát trọng tâm của nó là cùng một mô hình cơ bản có thể nhận được những điểm số khác nhau đáng kể khi vận hành thông qua các cách khai thác khác nhau. Bản tóm tắt báo cáo khoảng cách lên tới 16 điểm phần trăm do harness trong các so sánh bảng xếp hạng được quan sát.

Cơ sở lý luận của khách hàng có ba phần:

- 1 Harness thiết kế hiện đóng góp đáng kể vào agent hiệu suất. Chỉ riêng khả năng của mô hình không xác định liệu agent có thể hoàn thành nhiệm vụ đầu cuối hay không.
- 2 Sự đóng góp nhân quả của các lựa chọn harness cá nhân là không rõ ràng. Bảng xếp hạng công khai so sánh các hệ thống hoàn chỉnh, nhưng chúng thường không tách biệt tác động của prompt cấu trúc, công cụ, quản lý bối cảnh, thử lại hoặc xác minh.
- 3 Các nhóm xây dựng agent phải đối mặt với quyết định thực tế giữa tự xây dựng và áp dụng giải pháp có sẵn. Các tổ chức cần bằng chứng để quyết định liệu harness đã thiết lập có đủ hay lớp thực thi tùy chỉnh tạo ra đủ giá trị để biện minh cho chi phí kỹ thuật và bảo trì.

Do đó, dự án vừa là một bài tập kỹ thuật vừa là một nghiên cứu thực nghiệm có kiểm soát. Xây dựng một harness chức năng là cần thiết, nhưng sự đóng góp mạnh mẽ hơn là bằng chứng giải thích lý do tại sao nó hoạt động khác biệt.

## Phạm vi, mục tiêu và loại trừ chính thức

Bản tóm tắt dự án bằng văn bản thiết lập phạm vi thử nghiệm sau:

- sử dụng Terminal-Bench 2.1 làm đơn benchmark;
- giữ một mô hình không đổi trong các phép so sánh;
- so sánh hệ thống tùy chỉnh với hai harness đã được thiết lập;
- sử dụng tập hợp con phát triển 20 nhiệm vụ cố định để lặp lại;
- dành toàn bộ 89-nhiệm vụ benchmark để chấm điểm cuối cùng sau khi thiết kế bị đóng băng;
- xây dựng harness tối thiểu cần thiết để chạy các tác vụ thay vì một framework agent tổng quát;
- sửa đổi mỗi lần một đòn bẩy thiết kế;

- ghi lại độ chính xác, mã thông báo và chi phí cho mỗi lần chạy; và
- tích hợp harness tùy chỉnh với \_\_Harbor dưới dạng agent tùy chỉnh.

Bản tóm tắt rõ ràng loại trừ:

- Tinh chỉnh mô hình;
- so sánh giữa các mô hình khác nhau;
- viết các nhiệm vụ benchmark mới; và
- xây dựng một nền tảng agent rộng rãi, có mục đích chung.

Mục tiêu chính thức là tái tạo khoảng cách hiệu suất khai thác bằng cách sử dụng hai hệ thống đã thiết lập, triển khai harness tùy chỉnh tương thích với Harbor, thực hiện lặp lại có kiểm soát trên prompt, công cụ, ngữ cảnh hoặc thử lại thiết kế, hoạt động tốt hơn ít nhất một harness đã thiết lập trên tập hợp con phát triển, sau đó kiểm tra xem cải tiến có chuyển sang benchmark đầy đủ hay không.

Kết quả đầu ra dự kiến là:

- bảng điểm so sánh harness tùy chỉnh và các harness đã được thiết lập trong cùng điều kiện model và nhiệm vụ;
- kho lưu trữ harness đang hoạt động;
- nhật ký thay đổi kết nối từng thay đổi thiết kế với kết quả đo được; và
- gửi bảng xếp hạng công khai Terminal-Bench.

Việc gửi bảng xếp hạng hiện phải được coi là kết quả dự định thay vì lời hứa vô điều kiện. Các yêu cầu chạy lặp lại, cấp vốn API, quyền truy cập mô hình, giới hạn cơ sở hạ tầng, phê duyệt xuất bản và giao thức cuối cùng vẫn yêu cầu xác nhận.

## Kỳ vọng của môn học và người hướng dẫn

Tài liệu khởi động 36127 bổ sung các yêu cầu ảnh hưởng đến cách thức phân phối Dự án 15:

- công việc phải áp dụng kiến thức khóa học trước đó và có tính chất quan trọng hoặc có tính đổi mới;
- đội nên có năm hoặc sáu học sinh;
- mỗi thành viên dự kiến sẽ đóng góp ít nhất chín giờ mỗi tuần trong 12 tuần;
- nhóm nên gặp cố vấn của mình khoảng 30 phút mỗi tuần giảng dạy;
- đóng góp mã phải được quản lý thông qua Git;
- Slack nên được sử dụng cho giao tiếp nhóm và cố vấn trực quan;
- công việc, quyết định, thách thức và bằng chứng phải được theo dõi một cách chủ động; và
- dự án phải được ghi lại và trình bày cho khán giả học thuật hoặc chuyên nghiệp.

Các báo cáo cá nhân hàng tuần có ảnh hưởng dù không phải là một hạng mục đánh giá riêng biệt. Chúng đóng góp vào Hệ số Đóng góp Cá nhân dựa trên mức độ hoàn thành nhiệm vụ, chất lượng bằng chứng, sự tham gia họp và giao tiếp, phản tư, lập kế hoạch và tính chuyên nghiệp. Với dự án này, bằng chứng hữu ích gồm commit, cấu hình thí nghiệm, kết quả chạy thử, phân tích trajectory, nhận xét phản biện, biên bản họp, tính toán chi phí và các quyết định kỹ thuật bằng văn bản.

Các slide khởi đầu chứa đựng sự mâu thuẫn chưa được giải quyết về việc liệu các đánh giá chính là bài nộp của nhóm hay cá nhân. Các trang đánh giá Canvas có thẩm quyền và lời khuyên của người cố vấn hoặc điều phối viên chủ đề phải được kiểm tra trước khi hoàn thành trách nhiệm.

## Tại sao đây là chủ đề hiện tại và có động lực cao

Dự án 15 phù hợp chặt chẽ với sự chuyển đổi vào năm 2026 từ LLM đàm thoại sang hệ thống agent có thể thực thi. Trong các hệ thống này, mô hình hoạt động thông qua lớp phần mềm kiểm soát các công cụ, bối cảnh, trạng thái, quyền, thực thi, phản hồi, phục hồi và xác minh. Những cải tiến đối với lớp đó có thể thay đổi khả năng mà không cần đào tạo lại mô hình.

Một số diễn biến gần đây chứng tỏ động lực của chủ đề này:

- Terminal-Bench 2.1, được phát hành vào tháng 5 năm 2026, đã sửa 28 trong số 89 tác vụ và đưa ra xác thực benchmark liên tục. Điểm số của mô hình tác nhân đại diện đã thay đổi nhiều điểm phần trăm sau khi điều chỉnh, cho thấy cơ sở hạ tầng đánh giá ảnh hưởng đáng kể đến khả năng được báo cáo.
- Harness-Bench, được phát hành vào tháng 5 năm 2026, trực tiếp đánh giá harness hiệu ứng trên các phần phụ trợ của mô hình. Trên 5.194 quỹ đạo và 106 nhiệm vụ trong hộp cát, nó báo cáo sự khác biệt đáng kể về mức độ hoàn thành, hiệu quả, chất lượng quy trình và hành vi lỗi giữa model-harness cấu hình.
- TUA-Bench, được phát hành vào tháng 6 năm 2026, mở rộng đánh giá terminal agent lên 120 nhiệm vụ gồm hoạt động số thông thường, công việc khoa học và quy trình kỹ thuật. Kết quả mạnh nhất được báo cáo vẫn chưa đạt độ tin cậy hoàn toàn.
- Task Alignment Benchmark, bắt nguồn từ Terminal-Bench 2.1, cho thấy rằng hiệu suất hoàn thành nhiệm vụ cao không đảm bảo rằng agent phân biệt chính xác các hướng dẫn môi trường có liên quan với các yếu tố gây phân tâm gây hiểu lầm.
- Các nhà cung cấp agent chính hiện xuất bản hướng dẫn kỹ thuật chuyên dụng về khai thác lâu dài, tính liên tục của bối cảnh, xác minh, phối hợp đa tác nhân và phát triển phần mềm tự động.
- Các đại lý mã hóa thương mại ngày càng quảng cáo công việc từ đầu đến cuối như phân tích kho lưu trữ, triển khai tính năng, kiểm tra, di chuyển, đánh giá và thực thi tác vụ nền thay vì chỉ hoàn thành mã.

Những phát triển này hỗ trợ một kết luận chắc chắn: năng lực của agent nên được phân tích ở cấp cấu hình model-harness, không chỉ quy cho model. Dự án 15 trực tiếp nghiên cứu vấn đề cấp hệ thống mới nổi này.

Các lĩnh vực ứng dụng trong thế giới thực

| Khu vực ứng dụng                 | Ví dụ agent công việc                                                                                  | Harness yêu cầu khả năng                                                                                   | Rủi ro hoạt động chính                                                       |
|----------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Kỹ thuật phần mềm                | Sửa lỗi, triển khai tính năng, cấu trúc lại mã, viết bài kiểm tra, xem xét các thay đổi                | Bối cảnh kho lưu trữ, công cụ chỉnh sửa, thực hiện kiểm tra, nhận thức về Git, công cụ xác minh            | Những thay đổi không chính xác khi vượt qua các bài kiểm tra chưa hoàn chỉnh |
| DevOps và độ tin cậy             | Chẩn đoán sự cố, kiểm tra nhật ký, cập nhật cấu hình triển khai, xác thực khôi phục                    | Truy cập thiết bị đầu cuối an toàn, quyền hạn chế, khôi phục, ghi nhật ký sự kiện, phê duyệt của con người | Lệnh gián đoạn dịch vụ hoặc phá hoại                                         |
| Kỹ thuật dữ liệu                 | Sửa chữa đường ống, kiểm tra lược đồ, thực hiện chuyển đổi, xác thực chất lượng dữ liệu                | Kiểm soát công việc trong thời gian dài, đầu ra có cấu trúc, kiểm tra dữ liệu, khôi phục trạng thái        | Tham nhũng thảm lạng hoặc dữ liệu xuôi dòng không hợp lệ                     |
| An ninh mạng                     | Phân tích các lỗ hổng, tái tạo các hoạt động khai thác trong hộp cát, kiểm tra các biện pháp khắc phục | Cách ly, chính sách mạng nghiêm ngặt, đường kiểm tra, hạn chế về công cụ an toàn                           | Sử dụng sai, lộ dữ liệu hoặc thực thi không an toàn                          |
| Máy tính khoa học                | Định cấu hình môi trường, tái tạo phân tích, chạy mô phỏng, thu thập hiện vật                          | Quản lý phụ thuộc, môi trường có thể tái tạo, xuất xứ, kiểm soát tài nguyên                                | Kết quả không thể tái tạo hoặc bằng chứng bị đặt                             |
| Hoạt động mô hình và ML          | Chuẩn bị bộ dữ liệu, khởi chạy đánh giá, so sánh các lần chạy, chẩn đoán lỗi                           | Cấu hình thử nghiệm, giới hạn chi phí, thu thập số liệu, khôi phục điểm kiểm tra                           | Trôi cấu hình và so sánh không hợp lệ                                        |
| Di chuyển hệ thống kế thừa       | Dịch hoặc hiện đại hóa các cơ sở mã lớn và liên tục xác thực hành vi                                   | Phân rã, trạng thái nhiều phiên, kiểm soát công việc song song, kiểm tra hồi quy                           | Hồi quy hành vi trên nhiều tệp                                               |
| Tự động hóa quy trình kinh doanh | Vận hành các ứng dụng dòng lệnh, tạo báo cáo, di chuyển các tạo phẩm đã được xác thực                  | Các hành động đã nhập, kiểm tra chính sách, phê duyệt, ranh giới nhận dạng và quyền                        | Hành động trái phép hoặc trách nhiệm giải trình yếu kém                      |

Terminal-Bench không chứng tỏ được sự sẵn sàng cho tất cả các cơ sở sản xuất này. Nó cung cấp một proxy được kiểm soát để lập kế hoạch, thực hiện, giải thích phản hồi, sản xuất tạo tác và các hành vi xác minh mà nhiều người trong số họ yêu cầu.

Giá trị của bên liên quan và những quyết định thiết thực

Dự án có thể tạo ra bằng chứng cho bốn quyết định quan trọng.Xây dựng so với áp dụng: Một nhóm có thể thích harness đã được thiết lập vì nó được duy trì, giàu tính năng và đã được tích hợp với các công cụ phổ biến. harness tùy chỉnh chỉ hợp lý nếu nó mang lại lợi ích có ý nghĩa về độ chính xác, độ tin cậy, chi phí, khả năng kiểm soát hoặc khả năng kiểm toán.Khả năng so với hiệu quả: Một harness đạt được một nhiệm vụ thành công trong khi tăng gấp đôi chi phí mã thông báo và thời gian chạy có thể không phù hợp cho các hoạt động thông thường. Do đó, kết quả phải thể hiện giới hạn độ chính xác-chi phí-thời gian chạy thay vì chỉ độ chính xác.Tự chủ so với kiểm soát: Các tác nhân mở có thể thích ứng linh hoạt nhưng các quy trình làm việc bị ràng buộc sẽ dễ kiểm tra hơn và có thể tránh được sự lặp lại hoặc hoàn thành sớm. Dự án có thể xác định khi nào quyền tự chủ được bổ sung sẽ gặp rủi ro hoạt động.Thiết kế chung so với thiết kế nhận biết nhiệm vụ: Các công cụ chung chuyển giao giữa các nhiệm vụ nhưng có thể cung cấp hướng dẫn yếu. Các công cụ chuyên dụng có thể giảm thiểu lỗi nhưng có thể phù hợp quá mức với tập hợp con phát triển. Thiết kế cuối cùng phải giữ nguyên nhiệm vụ chung trong khi vẫn nhận biết được giao diện.

Phân tích thử thách chi tiết

| Thử thách                              | Tại sao nó quan trọng                                                                         | Bằng chứng cần thu thập                                                                          | Giảm nhẹ cho Dự án 15                                                                                    |
|----------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| So sánh công bằng                      | Sự khác biệt về kiểu máy hoặc cấu hình có thể bị nhầm lẫn với hiệu ứng harness                | ID mô hình, cài đặt lý luận, điều khiển lấy mẫu, hàm băm prompt, môi trường và phiên bản harness | Cấu hình chạy bất biến và so sánh tác vụ được ghép nối                                                   |
| Kết quả ngẫu nhiên                     | Một lần chạy có thể phóng đại chiến thắng hoặc thua                                           | Biến thể thử lặp lại, thắng/thua theo cặp, khoảng tin cậy                                        | Phân tích độ nhạy và chính sách lặp lại đã được thỏa thuận trước                                         |
| Benchmark giá trị                      | Các thử nghiệm có thể không đầy đủ, không ổn định hoặc không đạt được thành công như mong đợi | Verifier hành vi, hồ sơ vấn đề nhiệm vụ, kết quả tranh chấp                                      | Ghim Terminal-Bench 2.1; phân loại benchmark lỗi riêng biệt                                              |
| Phát triển quá mức                     | Điều chỉnh lặp đi lặp lại trên 20 tác vụ có thể không được truyền tới tất cả 89               | Hiệu suất phát triển so với cuối cùng và phân bổ lỗi                                             | Đóng băng tập hợp con sớm và chỉ chạy toàn bộ sau khi đóng băng thiết kế                                 |
| Nhiều thay đổi đồng thời               | Nguồn gốc của sự cải thiện trở nên không rõ ràng                                              | Cấu hình và thay đổi được phiên bản                                                              | Thí nghiệm một đòn bẩy với các lần chạy lại kiểm soát và điều trị                                        |
| Chi phí và tính toán                   | Đánh giá đầy đủ và thử nghiệm lặp đi lặp lại có thể vượt quá nguồn lực sẵn có                 | Mã thông báo, chi phí của nhà cung cấp, thời gian chạy, tỷ lệ thất bại và thử lại                | Kiểm tra khối, đánh giá theo giai đoạn, dự báo ngân sách hàng tuần                                       |
| Mất bối cảnh                           | Lịch sử lâu dài có thể ẩn trạng thái hiện tại và làm cạn kiệt cửa sổ ngữ cảnh                 | Kích thước ngữ cảnh, hành động lặp lại, lỗi thiếu trạng thái                                     | Quan sát giới hạn cộng với trạng thái thu gọn rõ ràng                                                    |
| Xác minh yếu                           | Đại lý có thể tuyên bố thành công mà không cần kiểm tra kết quả yêu cầu                       | Phạm vi xác minh và tỷ lệ hoàn thành sai                                                         | Hoàn thành có kiểm soát xác minh và sửa chữa dựa trên bằng chứng                                         |
| Lỗi công cụ và trình phân tích cú pháp | Hành động mẫu có thể không đúng định dạng hoặc không rõ ràng                                  | Tỷ lệ hành động không hợp lệ, tỷ lệ sửa chữa, lượt lãng phí                                      | Lược đồ hành động được đánh máy nhỏ và phản hồi khắc phục ngắn gọn                                       |
| Sự cố cơ sở hạ tầng                    | Docker, Harbor, lỗi lưu trữ, kết nối mạng hoặc nhà cung cấp có thể làm sai lệch điểm số       | Tách biệt cơ sở hạ tầng và hồ sơ lỗi của nhà cung cấp                                            | Chính sách chạy lại được ghi lại và phân tách lỗi                                                        |
| An toàn và quyền hạn                   | Tác nhân đầu cuối có thể sửa đổi tệp, thực thi các lệnh không an toàn hoặc truy cập bí mật    | Hồ sơ phê duyệt và hành động bị từ chối                                                          | Cách ly container, đặc quyền tối thiểu, kiểm soát mạng, cổng phê duyệt                                   |
| Prompt chèn và căn chỉnh nhiệm vụ      | Văn bản môi trường có thể chứa các hướng dẫn không liên quan hoặc độc hại                     | Lỗi theo dõi tin hiệu và phân tâm trong quỹ đạo                                                  | Phân cấp hướng dẫn, bối cảnh nhận biết xuất xứ, xác nhận có chọn lọc                                     |
| Khả năng tái tạo                       | Model và harness thay đổi nhanh chóng khiến kết quả khó lặp lại                               | Git commit, dependency lock, phiên bản dataset, trajectory đã lưu                                | Ghim phiên bản, manifest bất biến, lưu giữ artifact                                                      |
| Tính khái quát                         | Cài tiến Terminal-Bench không được chuyển sang công việc hoặc mô hình khác                    | Hiệu suất theo loại nhiệm vụ và điều kiện biên rõ ràng                                           | Giới hạn các xác nhận quyền sở hữu và đề xuất xác nhận trên benchmark khác cho công việc trong tương lai |



## Tính khả thi và vị trí dự án được đề xuất

Dự án chỉ khả thi trong vòng một học kỳ nếu nhóm tuân theo phạm vi khai thác tối thiểu chính thức. Việc xây dựng lại Claude Code, Codex CLI, hoặc OpenHands sẽ là không thực tế. Thay vào đó, hệ thống tùy chỉnh sẽ triển khai một số ít thành phần rõ ràng:

- cấu hình thử nghiệm bất biến;
- một prompt và chính sách kiểm soát theo giai đoạn;
- một giao diện hành động đầu cuối an toàn;
- quan sát có giới hạn và quản lý bối cảnh;
- có cấu trúc trajectory và ghi nhật ký số liệu;
- hoàn thành kiểm soát xác minh; và
- một đường dẫn sửa chữa được hướng dẫn bằng chứng có giới hạn.

Quan điểm học thuật mạnh mẽ nhất không phải là “chúng tôi đã xây dựng một agent khác”. Đó là:

**Chúng tôi tiến hành một nghiên cứu có kiểm soát về cách các lựa chọn thiết kế harness ảnh hưởng đến độ chính xác, chi phí và độ tin cậy trên Terminal-Bench 2.1 khi model nền được giữ cố định.**

Cách định vị này vẫn có giá trị trong nhiều kết quả có thể xảy ra:

- Tùy chỉnh harness thắng: xác định thay đổi được kiểm soát nào đã tạo ra lợi ích và chi phí hoạt động của thay đổi đó.
- Các ràng buộc harness tùy chỉnh với chi phí thấp hơn: cho thấy rằng lớp thực thi đơn giản hơn có thể có lợi hơn về mặt kinh tế.
- harness tùy chỉnh thua: giải thích những khả năng đã thiết lập nào có vẻ quan trọng và tại sao việc xây dựng hệ thống tùy chỉnh có thể không hợp lý.
- Kết quả được trộn lẫn theo danh mục nhiệm vụ: xác định các điều kiện biên và đề xuất chiến lược tương lai nhận biết nhiệm vụ.
- Không thể kết luận trong phạm vi ngân sách: ghi lại các hạn chế về cơ sở hạ tầng và thống kê một cách trung thực và cung cấp một giao thức có thể tái tạo để tiếp tục.

## Các quyết định cần có sự xác nhận của người cổ vấn hoặc khách hàng

Cuộc thảo luận đầu tiên của người cổ vấn nên chuyển những điều chưa biết sau đây thành các quyết định bằng văn bản:

- 1 Mô hình, nhà cung cấp và cài đặt lý luận chính xác nào phải được giữ cố định?
- 2 Hai khai thác được thiết lập nào là đường cơ sở có thẩm quyền?
- 3 Tập hợp con phát triển gồm 20 nhiệm vụ có được nhóm cung cấp, cùng lựa chọn hoặc lựa chọn không?
- 4 Điều gì tạo nên “nhịp”: độ chính xác của quá trình phát triển, một lần chạy đầy đủ, các lần chạy đầy đủ lặp lại, hiệu suất được điều chỉnh theo chi phí hoặc việc gửi bảng xếp hạng được chấp nhận?
- 5 Số lần thử lặp lại dự kiến cho các yêu cầu phát triển và cuối cùng là bao nhiêu?
- 6 Có sẵn ngân sách tin dụng API, điện toán, lưu trữ và thời gian thực thi nào?
- 7 Lỗi cơ sở hạ tầng hoặc nhà cung cấp nào có thể được chạy lại?
- 8 Việc gửi bảng xếp hạng công khai có bắt buộc không nếu chi phí, quyền truy cập hoặc phê duyệt xuất bản ngăn cản việc đó?
- 9 Điều gì phải được trình bày tại cuộc họp tiến độ sớm Tuần 5?
- 10 Những hiện vật nào có thể được công khai và những quy tắc ghi nhận hoặc bảo mật nào được áp dụng?
- 11 Đề xuất, báo cáo tiến độ, trình bày và báo cáo cuối cùng của nhóm hay của cá nhân?



12 Tiêu chuẩn thống kê nào được mong đợi để khẳng định sự cải thiện?

## Bài báo 1: Terminal-Bench

### Vấn đề nghiên cứu

Bài viết Terminal-Bench lập luận rằng nhiều điểm chuẩn agent hiện tại là quá giả tạo hoặc không đủ khó để đo lường các tác nhân biên giới. Công việc đầu cuối là một lĩnh vực thử nghiệm có giá trị vì nó bao gồm các hoạt động thực tế từ công nghệ phần mềm, học máy, an ninh mạng, tính toán khoa học, quản trị hệ thống và tái tạo nghiên cứu.

### Benchmark được đề xuất

Terminal-Bench 2.0 chứa 89 nhiệm vụ được quản lý thủ công. Mỗi nhiệm vụ bao gồm:

- hướng dẫn bằng tiếng Anh;
- môi trường chứa đựng;
- các thử nghiệm kiểm tra trạng thái thùng chứa cuối cùng;
- giải pháp tham khảo do con người viết; và
- Hạn chế về thời gian và nguồn lực.

benchmark hướng đến kết quả. Nó thường đánh giá trạng thái cuối cùng thay vì yêu cầu agent tuân theo một chuỗi lệnh được quy định. Điều này cho phép nhiều giải pháp hợp lệ trong khi vẫn duy trì xác minh khách quan.

>Phát hiện được báo cáo: Trong đánh giá ban đầu, sự kết hợp giữa mô hình-tác nhân biên giới giải quyết được ít hơn 65% nhiệm vụ, trong khi các mô hình nhỏ hơn đáng kể lại hoạt động kém hơn nhiều. Do đó, benchmark đã có đủ độ khó để bộc lộ sự khác biệt trong hành vi agent và harness.

### Phân tích lỗi

Bài viết cung cấp một phân loại hữu ích về các hành vi ngăn cản việc hoàn thành:

- Lỗi về thông số kỹ thuật: agent vi phạm định dạng, phương thức, đường dẫn bắt buộc hoặc ràng buộc nhiệm vụ khác.
- Lặp lại bước: agent lặp lại một hành động không thành công mà không rút ra được kết quả.
- Chấm dứt sớm: agent dừng trước khi đưa ra hoặc xác nhận kết quả được yêu cầu.
- Ảo giác hoặc đoán: agent thay thế một câu trả lời không được hỗ trợ khi không có thông tin bắt buộc.
- Không có hoặc xác minh không liên quan: agent không quan sát thấy bằng chứng về các yêu cầu chức năng cốt lõi.
- Xác minh yếu: có kiểm tra nhưng không bao gồm các thuộc tính cần thiết để đảm bảo tính chính xác thực sự.
- Lý luận-hành động không khớp: agent tuyên bố hoặc lên kế hoạch cho một việc trong khi các mệnh lệnh và tạo phẩm của nó lại hiển thị một việc khác.
- Chế tạo dữ liệu hoặc thao túng người đánh giá: agent tạo ra hoặc thay đổi bằng chứng lẽ ra phải được đo lường hoặc lấy ra một cách hợp pháp.

Sự khác biệt giữa không xác minh và xác minh yếu là quan trọng. Một agent không bao giờ thực hiện kiểm tra có lỗi khác với agent thực hiện kiểm tra sơ sài và coi nhầm nó là bằng chứng.

### Terminal-Bench 2.1

Dự án 15 chỉ định Terminal-Bench 2.1 thay vì bản phát hành 2.0 gốc được mô tả trong bài báo. Phiên bản 2.1 giữ lại 89 tác vụ benchmark nhưng sửa 28 tác vụ bị ảnh hưởng bởi thay đổi dependency bên ngoài, giới hạn tài nguyên không phù hợp, điểm yếu của verifier và các vấn đề về độ bền vững. So sánh chính thức cho thấy các điều chỉnh này đã làm thay đổi đáng kể điểm số của một số tổ hợp model-harness.

>Phát hiện được báo cáo: Benchmark hiệu chỉnh có thể thay đổi độ chính xác đo được nhiều điểm phần trăm ngay cả khi agent và mô hình không thay đổi.

>Ý nghĩa của dự án: Nhóm phải ghim tập dữ liệu chính xác, phiên bản Harbor, phiên bản harness, mô hình, nỗ lực lập luận, chính sách tài nguyên, chính sách mạng, thời gian chờ và số lần dùng thử. Nếu không, thay đổi cấu hình có thể bị nhầm lẫn với cải tiến trong harness tùy chỉnh.

## Ý nghĩa đánh giá

Độ chính xác là kết quả chính, nhưng một nghiên cứu đáng tin cậy cũng phải báo cáo:

- thành công theo từng nhiệm vụ;
- độ không đảm bảo hoặc sự thay đổi qua các thử nghiệm lặp lại;
- tiêu thụ mã thông báo;
- trị giá;
- thời gian chạy;
- tỷ lệ lỗi và thời gian chờ;
- loại nhiệm vụ;
- các loại hư hỏng; và
- các trường hợp trong đó verifier có thể không thể hiện tính đúng đắn dự kiến.

## Hạn chế

Không benchmark thể hiện đầy đủ công việc chuyên môn thực sự. Terminal-Bench nhiệm vụ được sắp xếp theo vùng và có giới hạn thời gian, đồng thời hiệu suất có thể phụ thuộc vào mức độ tiếp xúc với đào tạo mô hình, cơ sở hạ tầng, tính khả dụng của mạng và phạm vi bao phủ verifier. Điểm chuẩn công cộng cũng phải đối mặt với rủi ro ô nhiễm và quá phù hợp.

## Thí nghiệm ứng cử viên

So sánh chính sách hoàn thành cơ sở với chính sách kiểm soát xác minh:

- Điều khiển: agent có thể dừng khi tuyên bố hoàn thành.
- Điều trị: harness yêu cầu kiểm tra được quan sát có liên quan; các lần kiểm tra không thành công sẽ được trả lại mô hình cho một lần sửa chữa.

Đo lường độ chính xác của toàn bộ nhiệm vụ, hoàn thành sớm, phạm vi xác minh, mã thông báo, chi phí và thời gian chạy.

## Bài báo 2: SWE-agent

### Vấn đề nghiên cứu

SWE-agent hỏi liệu các giao diện được thiết kế cho nhà phát triển con người có phù hợp với các tác nhân mô hình ngôn ngữ hay không. Con người sử dụng các trình soạn thảo phong phú và có thể bỏ qua những thông tin không liên quan, giải thích tài liệu sâu rộng và phục hồi linh hoạt sau những sai sót. LLM có các giới hạn về bối cảnh, sự chú ý và hành động khác nhau.

### Agent-computer interface

Bài viết định nghĩa ACI là cả hai:

- các hành động mà mô hình có thể thực hiện; và
- sự thể hiện trạng thái môi trường và phản hồi được cung cấp sau những hành động đó.

Định nghĩa này rộng hơn danh sách công cụ. Nó bao gồm tài liệu lệnh, định dạng đầu ra, cửa sổ tệp, xử lý lịch sử, thông báo lỗi và rào chắn quy trình làm việc.

### Bốn nguyên tắc thiết kế

- 1 Các hành động phải đơn giản. Tên, thông số và hướng dẫn của công cụ phải dễ dàng để mô hình diễn giải.
- 2 Các hành động phải gọn nhẹ và hiệu quả. Một hoạt động phải đạt được tiến bộ có ý nghĩa mà không cần nhiều bước chuyển tiếp mong manh.
- 3 Phản hồi phải đầy đủ thông tin nhưng ngắn gọn. agent cần bằng chứng về những gì đã thay đổi, nhưng kết quả đầu ra không cần thiết sẽ làm tiêu tốn bối cảnh và làm xao lãng nhiệm vụ.
- 4 Rào chắn phải ngăn chặn việc truyền lỗi. Kiểm tra cú pháp và chỉnh sửa hạn chế giúp agent phát hiện và sửa lỗi sớm hơn.

### Kiến trúc

SWE-agent sử dụng vòng lặp quan sát-hành động-lý luận lặp đi lặp lại với các lệnh chuyên dụng cho:

- tìm kiếm tập tin và biểu tượng;
- xem tập tin bị giới hạn;
- chỉnh sửa;
- kiểm tra hoặc thực hiện chương trình; và
- quản lý bối cảnh/lịch sử.

Nó vẫn cho phép các lệnh shell thông thường khi được yêu cầu, nhưng các hoạt động kỹ thuật phần mềm thông thường nhận được giao diện thân thiện với mô hình.

### Đánh giá và cắt bỏ

Bài viết so sánh SWE-agent với các phương pháp truy xuất không tương tác và đường cơ sở tương tác chỉ dành cho shell trong khi vẫn giữ cố định mô hình cơ bản để so sánh có liên quan.

>Phát hiện được báo cáo: Trên tập hợp con cắt bỏ SWE-bench Lite, giao diện hoàn chỉnh đã đạt được cải tiến lớn so với đường cơ sở chỉ có shell. Nghiên cứu cũng phát hiện ra rằng các lựa chọn giao diện riêng lẻ đã thay đổi đáng kể tỷ lệ phân giải.

Các phát hiện cắt bỏ cụ thể bao gồm:

- kết quả tìm kiếm tóm tắt hoạt động tốt hơn giao diện lặp lại khuyến khích kiểm tra toàn diện;
- cửa sổ tệp bị chặn hoạt động tốt hơn khi hiển thị toàn bộ tệp;
- việc giữ lại một vài quan sát chi tiết cuối cùng tốt hơn việc giữ lại toàn bộ lịch sử;
- chỉnh sửa nhiều dòng nhỏ gọn là quan trọng;

- tự động linting cải thiện độ tin cậy chỉnh sửa; và
- các công cụ bổ sung có thể làm giảm hiệu suất khi chúng khuyến khích hành vi kém hiệu quả.

## Chi phí và bối cảnh

Nghiên cứu sử dụng ngân sách chi phí cho mỗi trường hợp. Điều này làm cho tính hiệu quả agent trở thành một phần của vấn đề thiết kế: một giao diện gây ra quá nhiều thao tác duyệt, xuất hoặc lặp lại có thể làm cạn kiệt ngân sách trước khi nhiệm vụ được giải quyết.

>Ý nghĩa của dự án: Đo lường số lượt, mã thông báo đầu vào, mã thông báo đầu ra và các hành động lặp lại, không chỉ đạt/không đạt. Một cải tiến harness có độ chính xác rất thấp trong khi chi phí tăng gấp đôi có thể không được bảo vệ.

## Hạn chế

Bài báo tập trung vào benchmark kỹ thuật phần mềm hơn là toàn bộ sự đa dạng miền của Terminal-Bench. Kết quả được thu thập với các model, prompt, công cụ và giới hạn chi phí cụ thể. Một ACI hữu ích cho model này có thể không hữu ích tương đương cho model khác.

## Thí nghiệm ứng cử viên

So sánh ba chiến lược quan sát:

- 1 đầu ra thiết bị đầu cuối thô đầy đủ và lịch sử đầy đủ;
- 2 đầu ra thô gần đây với cửa sổ lịch sử trượt; và
- 3 đầu ra bị giới hạn cộng với bản tóm tắt trạng thái nhiệm vụ liên tục và các hành động gần đây.

Đo lường độ chính xác, mức sử dụng mã thông báo, lỗi giới hạn ngữ cảnh, hành động lặp lại và khả năng phục hồi sau lỗi.

## Bài báo 3: OpenHands

### Vấn đề nghiên cứu

OpenHands trình bày một nền tảng mở để xây dựng các tác nhân phát triển phần mềm tổng quát. Nó giải quyết nhu cầu kết hợp các chính sách agent, quyền truy cập mô hình, thực thi mã, tương tác đầu cuối, duyệt, hộp cát, trạng thái, giao diện người dùng và đánh giá trong một hệ thống có thể mở rộng.

### Kiến trúc nền tảng

Nền tảng này thể hiện một số thành phần cũng xuất hiện trong Terminal-Bench harness:

- một sự trừu tượng của mô hình;
- logic agent;
- một thư viện các hành động và quan sát;
- thời gian chạy thực thi các hành động;
- một không gian làm việc biệt lập;
- lịch sử cuộc trò chuyện và sự kiện;
- logic điều khiển;
- giao diện ứng dụng; và
- benchmark bộ chuyển đổi.

Đại lý có thể viết mã, tương tác với dòng lệnh, sử dụng các khả năng của trình duyệt và hoạt động trong môi trường hộp cát. Nền tảng này cũng hỗ trợ so sánh nghiên cứu giữa các đại lý và điểm chuẩn.

### Mức độ liên quan với tư cách là harness

OpenHands là baseline hợp lý cho Dự án 15 vì Harbor hiện có tích hợp dành cho nó. Mức độ trưởng thành khiến OpenHands trở thành đối tượng so sánh có ý nghĩa, nhưng việc sử dụng công bằng đòi hỏi phải ghim phiên bản và cấu hình nó với cùng model cùng các ràng buộc benchmark tương đương.

>Phát hiện được báo cáo: OpenHands chứng minh rằng các tác nhân phần mềm tổng quát có thể được triển khai dưới dạng các chính sách mô-đun hoạt động thông qua hệ thống quan sát/hành động chung và thời gian chạy được kiểm soát.

>Ý nghĩa của dự án: Tái sử dụng các ranh giới kiến trúc chứ không phải toàn bộ nền tảng. Capstone tùy chỉnh harness chỉ được hiển thị chức năng cần thiết để kiểm tra các giả thuyết thiết kế đã chọn.

### Rủi ro về phạm vi quá mức

OpenHands bao gồm các khả năng không cần thiết đối với khả năng cung cấp tối thiểu của Dự án 15:

- ứng dụng đồ họa;
- duyệt rộng;
- nhiều giao diện người dùng ứng dụng;
- phối hợp đa tác nhân phức tạp;
- tích hợp triển khai sản xuất; và
- hệ sinh thái công cụ đa năng rộng lớn.

Cố gắng tái tạo những đặc điểm này sẽ tiêu tốn cả học kỳ mà không trả lời được câu hỏi nghiên cứu trọng tâm.

### Hạn chế

Bài viết OpenHands ban đầu đánh giá một nền tảng rộng rãi trên nhiều nhiệm vụ và tác nhân. Đây không phải là một nghiên cứu có kiểm soát đối với từng thành phần harness riêng lẻ. Một số hiệu suất không thể tách rời khỏi các mô hình, lời nhắc, công cụ và bộ điều hợp benchmark được sử dụng tại thời điểm đánh giá.

## Thí nghiệm ứng cử viên

Sử dụng OpenHands làm đường cơ sở đã được thiết lập và so sánh hành vi ở cấp độ quỹ đạo của nó với harness tùy chỉnh trên cùng các nhiệm vụ phát triển:

- số lượng và loại hành động;
- tỷ lệ các hành động làm thay đổi trạng thái nhiệm vụ;
- lệnh lặp lại hoặc thất bại;
- hành vi xác minh;
- quay trước khi hoàn thành; và
- chi phí cho mỗi nhiệm vụ thành công.

Phân tích có thể cho thấy vì sao harness tùy chỉnh thắng hoặc thua thay vì chỉ báo cáo độ chính xác.



## Bài báo 4: Agentless

### Vấn đề nghiên cứu

Agentless thách thức giả định rằng các tác nhân tự trị ngày càng phức tạp luôn là cách tiếp cận tốt nhất cho các vấn đề kỹ thuật phần mềm. Các tác nhân tự trị có thể tốn kém, khó tái tạo, dễ bị sai lệch và khó phân tích.

### Quy trình làm việc ba giai đoạn

Agentless sử dụng đường dẫn bị ràng buộc:

1. Bản địa hóa: xác định các tệp, lớp, chức năng và vị trí chỉnh sửa có liên quan.
2. Sửa chữa: tạo nhiều bản vá ứng cử viên cho các vị trí đã chọn.
3. Xác thực bản vá: sử dụng các bài kiểm tra hồi quy và tái tạo được tạo để lọc và xếp hạng các ứng viên.

Mô hình thực hiện việc tạo tập trung trong các giai đoạn này thay vì tự do lựa chọn các công cụ và hành động trong tương lai ở mỗi lượt.

### Những phát hiện chính

>Phát hiện được báo cáo: Trên SWE-bench Lite, Agentless đã báo cáo độ phân giải 32% với chi phí tương đối thấp, vượt trội so với các phương pháp tiếp cận dựa trên tác nhân nguồn mở được so sánh trong nghiên cứu đó.

Sự cắt bỏ của nó cho thấy rằng:

- bản địa hóa theo thứ bậc làm giảm ngữ cảnh trong khi vẫn giữ lại mã có liên quan;
- nhiều vị trí và mẫu bản vá cải thiện cơ hội tìm được cách sửa chữa chính xác;
- lựa chọn ứng viên là một trở ngại lớn về hiệu quả hoạt động;
- kiểm tra hồi quy cải thiện việc lựa chọn;
- các thử nghiệm tái tạo được tạo ra mang lại sự cải tiến đáng kể hơn nữa; và
- nhiều mẫu hơn cuối cùng mang lại giá trị thực tế giảm dần.

Các tác giả cũng xác định các nhiệm vụ benchmark có vấn đề với thông tin giải pháp bị thiếu, sai lệch hoặc bị rò rỉ và đề xuất benchmark được lọc.

### Sự liên quan đến Dự án 15

Agentless cung cấp hai bài học quan trọng:

1. Quy trình làm việc hẹp, có cấu trúc là cơ sở đáng tin cậy, không chỉ đơn thuần là một nguyên mẫu đơn giản hóa.
2. Việc xác minh và lựa chọn có thể đóng góp nhiều hơn việc lập kế hoạch không bị ràng buộc.

Capstone harness có thể kết hợp những bài học này mà không trở nên hoàn toàn không có tác nhân. Một vòng lặp tối thiểu có thể hạn chế hành vi:

```
Inspect -> Plan -> Execute -> Verify -> Repair once -> Finish
```

### Hạn chế

Agentless đánh giá cách giải quyết vấn đề trong kho mã thay vì các tác vụ đầu cuối đa dạng. Quy trình xác thực-sửa chữa-bản địa hóa của nó không được chuyển trực tiếp sang các tác vụ như cấu hình hệ thống, bảo mật, xử lý dữ liệu hoặc tái tạo nghiên cứu.

### Thí nghiệm ứng cử viên

So sánh:

- một vòng lặp kiểu ReAct có kết thúc mở; và
- chính sách kiểm tra-kế hoạch-thực thi-xác minh-sửa chữa theo từng giai đoạn.

Giữ các công cụ và mô hình không đổi. Đánh giá thành công, lượt quay, chi phí, hành động lặp lại và phân bổ danh mục thất bại.

## Bài báo 5: OpenHands Software Agent SDK

### Vấn đề nghiên cứu

Bài viết OpenHands SDK mô tả các bài học từ việc thiết kế lại một nền tảng nghiên cứu nguyên khối thành một nền tảng có thể kết hợp được cho các đại lý phần mềm đáng tin cậy. Khi hệ thống agent phát triển, thời gian chạy, ứng dụng, trạng thái và logic công cụ được kết hợp chặt chẽ khiến các thử nghiệm khó tái tạo và khó chẩn đoán lỗi.

### Bốn nguyên tắc kiến trúc

- 1 Cách ly tùy chọn: hỗ trợ thực thi cục bộ khi thích hợp trong khi vẫn duy trì hoạt động thực thi trong hộp cát để đảm bảo an toàn và kiểm soát tài nguyên.
- 2 Các thành phần không có trạng thái và một trạng thái có thể thay đổi có thẩm quyền: các thành phần được xây dựng không thể thay đổi làm giảm độ lệch cấu hình, trong khi một bản ghi trạng thái cho phép khôi phục.
- 3 Phân tách trách nhiệm nghiêm ngặt: tách agent cốt lõi khỏi giao diện người dùng và ứng dụng.
- 4 Gói có thể kết hợp và các thành phần được gói: cho phép các công cụ, không gian làm việc, mô hình và ứng dụng thay đổi thông qua giao diện rõ ràng.

### Trạng thái bắt nguồn từ sự kiện

Thay vì coi nhật ký là đầu ra phụ, SDK ghi lại các chuyển đổi trạng thái dưới dạng sự kiện. Điều này hỗ trợ:

- phát lại xác định;
- gỡ lỗi;
- phục hồi sau khi bị gián đoạn;
- khả năng kiểm toán;
- phân tích trajectory; và
- so sánh giữa các phiên bản hệ thống.

>Phát hiện được báo cáo: Bài viết báo cáo tỷ lệ thất bại do hệ thống giảm 61% sau khi kiến trúc được thiết kế lại, với chi phí phục hồi và duy trì trạng thái thấp khi so sánh trong quá trình sản xuất.

### Phân tách lỗi

Bài báo phân biệt các hư hỏng gây ra bởi:

- cơ sở hạ tầng và điều phối;
- logic SDK nội bộ; và
- nhà cung cấp mô hình bên ngoài.

Dự án 15 nên thêm benchmark/lỗi nhiệm vụ vào một danh mục riêng. Nếu không có sự phân tách này, thời gian chờ do Docker hoặc lỗi API gây ra có thể bị tính không chính xác là lỗi lý do.

### Sự liên quan đến Dự án 15

harness tùy chỉnh sẽ tách biệt:

- cấu hình thử nghiệm bất biến;
- agent chính sách;
- prompt mẫu;
- giao diện đầu cuối;
- quản lý bối cảnh;
- chính sách xác minh;
- trajectory nhà văn; và

- Harbor adapter.

Sự phân rã này cho phép nhóm thay đổi một đôn bầy thử nghiệm mà không làm thay đổi hành vi không liên quan.

## Hạn chế

Bài báo SDK tập trung vào độ tin cậy trong production và một hệ sinh thái software agent rộng lớn. Project 15 không yêu cầu dịch vụ production, truy cập đa người thuê, giao diện đồ họa hay nền tảng triển khai từ xa. Phân tích bảo mật vẫn chưa hoàn hảo, và kết quả production được báo cáo không trực tiếp chứng minh mức cải thiện trên Terminal-Bench.

## Thí nghiệm ứng cử viên

Kiểm tra xem trạng thái thu gọn dựa trên sự kiện có cải thiện khả năng phục hồi hay không:

- kiểm soát sử dụng lịch sử trò chuyện thông thường;
- xử lý lưu trữ các sự kiện rõ ràng và tạo ra một bản tóm tắt trạng thái nhỏ gọn sau ngưỡng mã thông báo.

Đánh giá lỗi ngữ cảnh, tiếp tục thành công sau thời gian dài xuất ra, mã thông báo và độ chính xác.

## Tổng hợp liên bài báo

| Nguồn          | Đóng góp chính                                                              | Harness đòn bẩy                                        | Bằng chứng                                                                            | Hạn chế                                   | Dự án sử dụng                                                        |
|----------------|-----------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------|
| Terminal-Bench | Thiết bị đầu cuối được xác minh kết quả khó khăn benchmark và phân loại lỗi | Xác minh, dừng, phân tích lỗi                          | 89 nhiệm vụ bộc lộ hiệu suất đáng kể chưa được giải quyết và các lỗi hành vi tái diễn | Giới hạn benchmark và verifier công khai  | benchmark chính và khung phân loại lỗi                               |
| SWE-agent      | Giao diện tác nhân-máy tính được điều chỉnh theo các giới hạn của mô hình   | Công cụ, phản hồi, bối cảnh, lan can                   | Việc cắt bỏ giao diện ảnh hưởng đáng kể đến hiệu suất của mô hình cố định             | Nhiệm vụ chủ yếu của kho phần mềm         | Nguồn ưu tiên cao nhất cho các thử nghiệm công cụ và bối cảnh        |
| OpenHands      | Nền tảng tác nhân phần mềm mở, chung                                        | Tách thời gian chạy, hành động/quan sát, hộp cát       | Thể hiện sự tích hợp nền tảng đại lý rộng rãi                                         | Quá rộng để tái tạo trong Capstone        | Tham chiếu cơ sở và kiến trúc được thiết lập                         |
| Agentless      | Thay thế theo giai đoạn đơn giản cho các đại lý tự trị                      | Các ràng buộc về quy trình làm việc, xác thực ứng viên | Bảo cáo mạnh mẽ về độ chính xác/chi phí và việc cắt bỏ xác minh                       | Đường ống dành riêng cho tên miền         | Bằng chứng về harness theo giai đoạn tối thiểu và trọng tâm xác minh |
| OpenHands SDK  | Kiến trúc sản xuất theo mô-đun, có nguồn gốc từ sự kiện                     | Trạng thái, nhật ký, tính bất biến, phục hồi           | Bảo cáo cải thiện độ tin cậy sau khi thiết kế lại                                     | Trọng tâm sản xuất vượt quá phạm vi dự án | Ranh giới thành phần có thể tái tạo và phân tách lỗi                 |

### Chủ đề 1: Harness thiết kế có ý nghĩa thực nghiệm

SWE-agent cho thấy các quyết định giao diện có thể cải thiện hiệu suất khi model được giữ cố định. Terminal-Bench chứng minh rằng agent thất bại vì các nguyên nhân hành vi mà harness có thể tác động. Điều này trực tiếp hỗ trợ tiền đề của Project 15.

### Chủ đề 2: Nhiều quyền tự chủ hơn và nhiều công cụ hơn không phải lúc nào cũng tốt hơn

SWE-agent báo cáo hành vi không hiệu quả với một số giao diện tìm kiếm và Agentless thể hiện sức mạnh của quy trình bị ràng buộc. Nhóm nên biện minh cho mọi khả năng được bổ sung bằng một lỗi có thể quan sát được mà nhóm dự định giảm thiểu.

### Chủ đề 3: Xác minh là cơ chế kiểm soát

Terminal-Bench xác định xác minh thiếu và yếu là chế độ lỗi. Agentless cho thấy rằng các bài kiểm tra cải thiện việc lựa chọn ứng viên. Do đó, việc xác minh phải là một phần của chính sách agent chứ không phải là bước báo cáo cuối cùng.

### Chủ đề 4: Bối cảnh là nguồn tài nguyên thử nghiệm có hạn

Toàn bộ đầu ra và lịch sử thiết bị đầu cuối không miễn phí. Chúng làm tăng mức tiêu thụ mã thông báo, giữ lại trạng thái lỗi thời và có thể giảm số lượt quay hữu ích. Các quan sát giới hạn của SWE-agent và quản lý trạng thái rõ ràng của OpenHands SDK hỗ trợ thử nghiệm bối cảnh nhỏ gọn.

### Chủ đề 5: Khả năng tái tạo đòi hỏi kiến trúc

Cấu hình bất biến, trajectory có cấu trúc, ghim phiên bản và phân tách lỗi không chỉ là công việc quản trị. Chúng cần thiết để quy khác biệt điểm số cho một thay đổi của harness.

## Hàm ý đối với harness tùy chỉnh của nhóm

### Kiến trúc tối thiểu được đề xuất

|                                  |
|----------------------------------|
| Harbor task instruction          |
|                                  |
| Immutable run configuration      |
|                                  |
| Prompt and staged agent policy   |
|                                  |
| Terminal action parser           |
|                                  |
| Harbor BaseEnvironment execution |
|                                  |
| Bounded observation processor    |
|                                  |
| Event/trajectory record          |
|                                  |
| Verification gate                |
|                                  |
| Limited error-aware repair       |
|                                  |
| Completion or budget exhaustion  |

### Ranh giới thành phần được đề xuất

| Thành phần                      | Trách nhiệm                                                         | Không được kiểm soát                          |
|---------------------------------|---------------------------------------------------------------------|-----------------------------------------------|
| Cấu hình                        | Bộ dữ liệu, mô hình, giới hạn, phiên bản prompt, cài đặt đơn<br>bầy | Quyết định về thời gian chạy                  |
| Agent chính sách                | Quyết định giai đoạn tiếp theo và yêu cầu hành động tiếp<br>theo    | Thực thi lệnh trực tiếp                       |
| Prompt chiến lược               | Kết xuất nhiệm vụ, trạng thái, tài liệu công cụ và hướng dẫn        | Lưu trữ trạng thái trajectory có thể thay đổi |
| Giao diện công cụ               | Phân tích và xác thực các hành động đầu cuối được yêu cầu           | Quyết định xem nhiệm vụ đã hoàn thành chưa    |
| Bộ chuyển đổi môi trường        | Thực hiện thông qua Harbor và trả về kết quả                        | Viết lại đầu ra mô hình                       |
| Trình quản lý bối cảnh          | Giữ lại trạng thái hiện tại và giới hạn bằng chứng gần đây          | Thay đổi cấu hình benchmark                   |
| Chính sách xác minh             | Yêu cầu bằng chứng và kích hoạt sửa chữa có giới hạn                | Thay đổi chính thức verifier                  |
| Trajectory người ghi<br>nhật ký | Kiểm tra các sự kiện và số liệu                                     | Ảnh hưởng đến hành vi agent                   |

### Cố tình loại trừ từ phiên bản đầu tiên

- nhiều đại lý hợp tác;
- Tinh chỉnh mô hình;
- học tăng cường;
- tìm kiếm internet không hạn chế;
- giao diện người dùng đồ họa;
- định tuyến mô hình động;
- một thị trường plugin chung;
- bộ nhớ đa tác vụ liên tục; và
- giải pháp mã hóa cứng dành riêng cho nhiệm vụ.

Những tính năng này chỉ có thể được xem lại nếu harness tối thiểu ổn định và lỗi quan sát được không thể giải quyết bằng một thay đổi nhỏ hơn.

## Biến thực nghiệm

### Thí nghiệm 1: cấu trúc Prompt

- Điều khiển: hướng dẫn hoàn thành tự động trực tiếp.
- Xử lý: quy trình làm việc kiểm tra-kế hoạch-thực hiện-xác minh rõ ràng.
- Giả thuyết: Hướng dẫn theo giai đoạn giúp giảm lỗi đặc tả và hoàn thành sớm với chi phí mã thông báo nhỏ.

### Thí nghiệm 2: Kiểm tra công xác minh

- Kiểm soát: chấp nhận hoàn thành theo mô hình đã khai báo.
- Xử lý: yêu cầu phải tiến hành kiểm tra liên quan trước khi hoàn thành.
- Giả thuyết: việc kiểm tra xác minh cải thiện độ chính xác bằng cách giảm số lần hoàn thành chưa được xác minh.

### Thí nghiệm 3: Sửa chữa khi nhận biết lỗi

- Kiểm soát: không có hành động đặc biệt nào sau khi kiểm tra không thành công.
- Xử lý: trả lại bằng chứng lỗi và cho phép sửa chữa tập trung.
- Giả thuyết: một lần sửa chữa sẽ khôi phục được một tỷ lệ lỗi hữu ích mà không gây ra vòng lặp không kiểm soát được.

### Thí nghiệm 4: Quản lý bối cảnh

- Kiểm soát: toàn bộ lịch sử và đầu ra thô.
- Phương pháp xử lý A: cửa sổ trượt các quan sát gần đây.
- Phương pháp điều trị B: tóm tắt trạng thái liên tục cộng với các hành động gần đây và kết quả đầu ra bị giới hạn.
- Giả thuyết: trạng thái nhỏ gọn làm giảm mã thông báo và sự lặp lại trong khi vẫn duy trì hoặc cải thiện độ chính xác.

### Thí nghiệm 5: Độ chi tiết của công cụ

- Điều khiển: một giao diện đầu cuối chung.
- Xử lý: các giao diện nhỏ gọn riêng biệt để tìm kiếm, kiểm tra tệp, chỉnh sửa và thực thi.
- Giả thuyết: các công cụ có cấu trúc giúp giảm các lệnh không hiệu quả và lỗi chỉnh sửa, nhưng việc lựa chọn công cụ quá mức có thể bù lại lợi ích.

### Quy tắc một đòn bẩy

Mỗi so sánh phải giữ tất cả các giá trị cấu hình khác không đổi. Nếu việc sửa lỗi cần thiết ảnh hưởng đến cả biện pháp kiểm soát và xử lý, hãy chạy lại cả hai điều kiện hoặc loại trừ các kết quả bị ảnh hưởng bằng cơ sở lý luận được ghi lại.

## Khung đánh giá

### Kết quả chính

- Độ chính xác của nhiệm vụ: tỷ lệ nhiệm vụ vượt qua verifier chính thức.

### Kết quả phụ

- mã thông báo đầu vào tổng số và mỗi nhiệm vụ;
- mã thông báo đầu ra tổng cộng và mỗi nhiệm vụ;
- chi phí tiền tệ;
- thời gian chạy đồng hồ treo tường;
- số vòng mô hình;
- số lượng hành động đầu cuối;
- đếm lại;
- tỷ lệ thời gian chờ;
- tỷ lệ lỗi cơ sở hạ tầng;
- phạm vi xác minh;
- tốc độ hành động lặp lại; và
- phân phối loại lỗi.

### Giao thức phát triển

- 1 Thống nhất với người cố vấn về tập hợp con phát triển chính xác gồm 20 nhiệm vụ.
- 2 Đóng băng và ghi lại tập hợp con trước khi tối ưu hóa harness.
- 3 Chạy nhóm khởi cơ sở hạ tầng gồm ba đến năm nhiệm vụ.
- 4 Tái tạo hai baseline harness đã được thiết lập với cùng một model.
- 5 Thiết lập đường cơ sở harness tùy chỉnh tối thiểu.
- 6 Thay đổi từng đòn bẩy thiết kế một lần.
- 7 Lặp lại các so sánh ranh giới nếu ngân sách cho phép.
- 8 Chọn thiết kế cuối cùng bằng cách sử dụng quy tắc đã được thống nhất trước thay vì ưu tiên chủ quan.
- 9 Đóng băng mã, cấu hình và phần phụ thuộc.
- 10 Chỉ chạy toàn bộ 89 tác vụ sau khi bị đóng băng.

### Phân tích thống kê

Vì kết quả của nhiệm vụ là các kết quả nhị phân được ghép nối nên hãy so sánh việc khai thác theo từng nhiệm vụ. Báo cáo:

- độ chính xác và số lượng nhiệm vụ thô;
- thắng, thua và hòa theo cặp;
- khoảng tin cậy;
- Thử nghiệm của McNemar khi quy trình lặp lại và cỡ mẫu phù hợp;
- độ nhạy cảm với các hư hỏng của cơ sở hạ tầng;
- thực hiện theo loại nhiệm vụ; và
- chi phí cho mỗi nhiệm vụ thành công.



Ý nghĩa thống kê không nên thay thế việc giải thích thực tế. Mức tăng độ chính xác nhỏ với mức tăng chi phí lớn có thể không phải là một cải tiến hữu ích.

**Bảng xếp hạng so với đánh giá học thuật**

Quá trình gửi Terminal-Bench 2.1 chính thức yêu cầu ít nhất năm lần thử cho mỗi nhiệm vụ để có được mục nhập bảng xếp hạng công khai. Điều này ngụ ý ít nhất 445 lần chạy khai thác tùy chỉnh trước các thử nghiệm cơ bản và phát triển. Nhóm phải nhận được xác nhận bằng văn bản về quyền truy cập mô hình, nguồn tài trợ API, cơ sở hạ tầng và kỳ vọng gửi trước khi coi việc gửi bảng xếp hạng là được đảm bảo.

**Phân loại lỗi**

| Danh mục                            | Định nghĩa hoạt động                                                        | Ví dụ harness phản hồi                                                           |
|-------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Đặc điểm kỹ thuật                   | Phương thức, đường dẫn, định dạng hoặc ràng buộc bắt buộc bị vi phạm        | Củng cố các ràng buộc và kiểm tra hiện vật trước khi hoàn thành                  |
| Lập lại                             | Mô hình hành động kém hiệu quả tương tự tái diễn mà không có bằng chứng mới | Phát hiện sự tương đồng về hành động và yêu cầu chẩn đoán sửa đổi                |
| Hoàn thành sớm                      | Agent dừng mà không đáp ứng hoặc kiểm tra các yêu cầu cốt lõi               | Áp dụng cổng xác minh                                                            |
| Ảo giác/đoán                        | Kết quả không được hỗ trợ thay thế cho bằng chứng còn thiếu                 | Yêu cầu xuất xứ bằng chứng và cấm hoàn thành không được hỗ trợ                   |
| Không xác minh                      | Không có kiểm tra cốt lõi có liên quan nào được quan sát                    | Yêu cầu kiểm tra có mục tiêu                                                     |
| Xác minh yếu                        | Kiểm tra không bao gồm các thuộc tính bắt buộc                              | Cung cấp danh sách kiểm tra xác minh hoặc lệnh kiểm tra chính thức nếu được phép |
| Lý trí-hành động không phù hợp      | Tuyên bố mâu thuẫn với các lệnh, lỗi hoặc tạo tác                           | Trả lại bằng chứng mâu thuẫn cho mô hình                                         |
| Lỗi ngữ cảnh                        | Trạng thái quan trọng bị mất hoặc đạt đến giới hạn ngữ cảnh                 | Sử dụng trạng thái bắt nguồn từ sự kiện nhỏ gọn                                  |
| Lỗi công cụ/trình phân tích cú pháp | Hành động mô hình không thể được diễn giải hoặc thực thi                    | Trả về lỗi lược đồ ngắn gọn và cho phép sửa lỗi                                  |
| Cơ sở hạ tầng                       | Docker, Harbor, lưu trữ hoặc điều phối không thành công                     | Loại trừ hoặc chạy lại theo chính sách cơ sở hạ tầng được ghi lại                |
| Nhà cung cấp                        | Xác thực, giới hạn tốc độ hoặc API mô hình không thành công                 | Ghi riêng và chỉ phát lại theo chính sách đã thỏa thuận                          |
| Benchmark/ __verifier               | Môi trường tác vụ hoặc verifier bị lỗi hoặc không ổn định                   | Lập tài liệu, cô lập và tham khảo ý kiến của người cố vấn thay vì âm thầm và lỗi |
| Cạn kiệt ngân sách                  | Đã đạt đến giới hạn mã thông báo, lượt, thời gian hoặc chi phí              | Lưu trạng thái cuối cùng và phân loại không có phần mở rộng ẩn                   |

## Khả năng tái tạo, chi phí và hiệu lực

### Bản ghi chạy bất biến

Mỗi lần dùng thử nên lưu trữ:

- ID thử nghiệm và thử nghiệm;
- dấu thời gian;
- Cam kết Git;
- Terminal-Bench dữ liệu và sửa đổi;
- Harbor phiên bản và cam kết nếu có;
- tên và phiên bản cơ bản/tùy chỉnh harness;
- mã định danh mô hình;
- nhà cung cấp;
- nỗ lực lý luận;
- kiểm soát nhiệt độ và lấy mẫu;
- prompt phiên bản và hàm băm;
- cấu hình công cụ;
- chính sách bối cảnh;
- chính sách thử lại;
- cài đặt thời gian chờ và tài nguyên;
- chính sách mạng;
- định danh nhiệm vụ;
- phần thưởng cuối cùng hoặc đạt/không đạt;
- mã thông báo, chi phí và thời gian chạy;
- chi tiết lỗi;
- phân loại hư hỏng; và
- trajectory/đường dẫn tạo tác.

### Kiểm soát chi phí

- chỉ sử dụng các lần chạy oracle để xác thực cơ sở hạ tầng chứ không phải để điều chỉnh agent;
- sử dụng ba đến năm nhiệm vụ để gỡ lỗi đường ống;
- tránh chạy lại các đường cơ sở đã thiết lập khi kết quả đã lưu vẫn tương thích với giao thức;
- thực thi các giới hạn về mã thông báo, thời gian và chi phí cho mỗi lần dùng thử;
- bảo quản mọi trajectory có thể sử dụng được;
- chỉ thực hiện các lần chạy đầy đủ sau khi thiết kế bị đóng băng;
- ước tính ngân sách thử nghiệm còn lại hàng tuần; và
- phân biệt chi phí được lưu trong bộ nhớ cache, ước tính và do nhà cung cấp báo cáo.

### Rủi ro hiệu lực nội bộ

- thay đổi nhiều hơn một tính năng harness;
- sử dụng cài đặt model khác nhau giữa các harness;

- chỉ chạy lại âm thầm các điều kiện không thành công;
- lựa chọn các nhiệm vụ phát triển sau khi quan sát kết quả;
- thay đổi cơ sở hạ tầng giữa các phương pháp điều trị;
- điều chỉnh prompt theo nhiệm vụ cụ thể; và
- loại trừ các hư hỏng không nhất quán.

### Rủi ro về độ giá trị ngoại tại

- kết quả có thể không được chuyển sang mô hình khác;
- tập hợp con 20 nhiệm vụ có thể không đại diện cho tất cả 89 nhiệm vụ;
- Terminal-Bench có thể không thể hiện đầy đủ công việc tương tác chuyên nghiệp;
- việc tiếp xúc với công chúng benchmark có thể ảnh hưởng đến mức độ quen thuộc của người mẫu; và
- mức cải thiện trên Terminal-Bench có thể không chuyển giao sang SWE-bench hoặc các agent benchmark khác.

### Rủi ro về độ giá trị cấu trúc

- các bài kiểm tra chính thức có thể bỏ qua các thuộc tính nhiệm vụ quan trọng;
- đạt/không đạt có thể che giấu tiến trình một phần;
- chi phí mã thông báo có thể được báo cáo khác nhau giữa các nhà cung cấp;
- thời gian chạy có thể bị chi phối bởi cơ sở hạ tầng thay vì chính sách harness; và
- xếp hạng trên bảng xếp hạng có thể khuyến khích việc tối ưu hóa không mang tính khái quát.

## Câu hỏi nghiên cứu và giả thuyết

### Câu hỏi nghiên cứu chính

Các lựa chọn thiết kế harness ảnh hưởng như thế nào đến độ chính xác, chi phí và độ tin cậy trên Terminal-Bench 2.1 khi model nền được giữ không đổi?

### Câu hỏi hỗ trợ

- 1 Quy trình làm việc kiểm tra-kế hoạch-thực thi-xác minh rõ ràng có tốt hơn prompt tự trị trực tiếp không?
- 2 Việc hoàn thành qua kiểm soát xác minh có làm giảm các tuyên bố thành công sớm hoặc không được hỗ trợ không?
- 3 Liệu một nỗ lực sửa chữa nhận biết lỗi có cải thiện độ chính xác đủ để biện minh cho các mã thông báo và thời gian chạy bổ sung của nó không?
- 4 Quản lý nhà nước gọn nhẹ có làm giảm sự lặp lại và sai sót trong bối cảnh mà không loại bỏ bằng chứng cần thiết không?
- 5 Các công cụ đầu cuối có cấu trúc có hoạt động tốt hơn giao diện shell chung không?
- 6 Loại hư hỏng nào bị ảnh hưởng mạnh nhất bởi mỗi lần thay đổi thiết kế?
- 7 Liệu một harness tùy chỉnh tối thiểu có thể hoạt động tốt hơn ít nhất một harness được thiết lập trên tập hợp con phát triển cố định và đánh giá đầy đủ không?

### Giả thuyết làm việc được đăng ký trước

- H1: Nhắc nhở theo giai đoạn sẽ làm giảm các lỗi do đặc điểm kỹ thuật và lỗi hoàn thành trước thời hạn.
- H2: Việc kiểm tra cổng xác minh sẽ cải thiện tỷ lệ đậu nhiều hơn là làm tăng chi phí.
- H3: Một lần sửa chữa dựa trên bằng chứng sẽ khắc phục các lỗi một cách hiệu quả; việc thử lại không hạn chế sẽ không cần thiết.
- H4: Tóm tắt trạng thái cộng với bằng chứng gần đây sẽ sử dụng ít mã thông báo hơn lịch sử đầy đủ mà không làm giảm độ chính xác.
- H5: Một bộ công cụ có cấu trúc nhỏ sẽ giảm thiểu lỗi chỉnh sửa và điều hướng, nhưng quá nhiều công cụ sẽ làm tăng lỗi lựa chọn.

## Phân bổ đọc sáu thành viên

| Thành viên   | Đọc sơ cấp                                          | Đầu ra bắt buộc                                                                    | Đánh giá chéo                                                  |
|--------------|-----------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Thành viên 1 | Terminal-Bench giấy và ghi chú phát hành 2.1        | Benchmark cấu trúc, phân loại lỗi, danh sách kiểm tra kiểm soát phiên bản          | Xem lại ghi chú về tính hợp lệ của điểm chuẩn của Thành viên 4 |
| Thành viên 2 | SWE-agent                                           | Nguyên tắc thiết kế ACI, bảng cắt bỏ, thí nghiệm giao diện ứng viên                | Xem lại ghi chú kiến trúc của Thành viên 5                     |
| Thành viên 3 | OpenHands giấy nền tảng                             | Kiến trúc cơ bản, Harbor câu hỏi so sánh, loại trừ phạm vi                         | Xem lại ghi chú về độ tái lập của Thành viên 6                 |
| Thành viên 4 | Agentless                                           | Quy trình làm việc tối thiểu, chiến lược xác thực, đối số về tính đơn giản/chỉ phí | Xem lại cách phân loại thất bại của Thành viên 1               |
| Thành viên 5 | OpenHands SDK                                       | Trạng thái, ghi nhật ký sự kiện, tính mô đun, phân tách lỗi                        | Xem xét đề xuất thử nghiệm của Thành viên 2                    |
| Thành viên 6 | Tổng hợp chéo giấy tờ và tài liệu chính thức Harbor | Thuật ngữ được chia sẻ, bảng bằng chứng, câu hỏi cố vấn, kiểm toán tham khảo       | Xem lại phân tích cơ bản của Thành viên 3                      |

Mỗi thành viên cần chuẩn bị:

- một bản tóm tắt một trang;
- ba phát hiện được báo cáo;
- hai hạn chế;
- hai tác động đối với Dự án 15;
- một thí nghiệm ứng cử viên;
- một câu hỏi dành cho người cố vấn; và
- tài liệu tham khảo trang hoặc phần chính xác.

Bài tập cặp:

- Cặp A - Thành viên 1 và 2: Harbor môi trường và các đường cơ sở đã được thiết lập.
- Cặp B - Thành viên 3 và 4: hành vi harness và agent tùy chỉnh.
- Cặp C - Thành viên 5 và 6: đánh giá, phân tích và ghi chép.

Các cặp nên luân phiên nhiệm vụ phụ ở các mốc Tuần 5, Tuần 7 và Tuần 9 để mọi thành viên đều có được kinh nghiệm về kỹ thuật, phân tích và giao tiếp.

## Thứ tự đọc khuyến nghị

- 1 Đánh giá tài liệu này- chia sẻ từ vựng và định hướng dự án.
- 2 Terminal-Bench giấy- benchmark hợp đồng, đánh giá và thất bại.
- 3 Terminal-Bench 2.1 tài liệu phát hành và chạy- tập dữ liệu và lệnh hiện tại.
- 4 SWE-agent- bằng chứng có mức độ ưu tiên cao nhất cho các thử nghiệm giao diện.
- 5 Agentless- bằng chứng về quy trình xác thực và quy trình làm việc theo giai đoạn tối thiểu.
- 6 OpenHands giấy nền tảng- đã thiết lập harness và tham chiếu nền tảng.
- 7 OpenHands SDK giấy- kiến trúc có thể tái tạo và đáng tin cậy.
- 8 Harbor agent tài liệu- tích hợp tác nhân tùy chỉnh cụ thể.

Cả nhóm nên đọc Terminal-Bench tóm tắt, xây dựng nhiệm vụ, đánh giá và phân tích thất bại. Sau đó, các thành viên được phân công sẽ hướng dẫn việc đọc sâu hơn và giảng dạy cho nhóm.

## Câu hỏi cố vấn ngay lập tức

- 1 Mô hình chính xác và cách lập luận nào phải cố định?
- 2 Nên sử dụng hai harness đã được thiết lập nào?
- 3 Khách hàng hoặc trường đại học có cung cấp tín chỉ API hoặc tính toán không?
- 4 Tập hợp con phát triển 20 nhiệm vụ có được cung cấp hay nhóm phải chọn và cố định nó?
- 5 Dự kiến có bao nhiêu thử nghiệm lặp lại trong quá trình phát triển và đánh giá cuối cùng?
- 6 “Đánh bại harness” có đề cập đến tập hợp con phát triển, một lượt chạy đầy đủ, các lượt chạy đầy đủ lặp lại hay một lần gửi bảng xếp hạng được chấp nhận không?
- 7 Những lỗi cơ sở hạ tầng nào có thể được chạy lại và việc chạy lại phải được báo cáo như thế nào?
- 8 Có được phép thay thế mẫu máy/nhà cung cấp nếu cấu hình ưu tiên không đủ khả năng chi trả?
- 9 Những sản phẩm nào được mong đợi tại các cuộc họp khách hàng Tuần 5, Tuần 7 và Tuần 9?
- 10 Áp dụng các quy tắc ghi nhận kho lưu trữ, xuất bản và bảng xếp hạng nào?

## Tài liệu tham khảo

- 1 Merrill, M. A., et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. <https://arxiv.org/abs/2601.11868>
- 2 Terminal-Bench Team. (2026). Terminal-Bench 2.1. <https://www.tbench.ai/news/terminal-bench-2-1>
- 3 Terminal-Bench Team. (2026). How to run Terminal-Bench 2.1. <https://www.tbench.ai/docs/run-terminal-bench-2-1>
- 4 Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. <https://arxiv.org/abs/2405.15793>
- 5 Wang, X., et al. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. <https://arxiv.org/abs/2407.16741>
- 6 Xia, C. S., et al. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. <https://arxiv.org/abs/2407.01489>
- 7 Wang, X., et al. (2026). The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. <https://arxiv.org/abs/2511.03690>
- 8 Harbor Framework. (2026). Agents: Using popular agents and integrating your own. <https://www.harborframework.com/docs/agents>
- 9 Harbor Framework. (2026). Terminus-2 reference agent. <https://www.harborframework.com/docs/agents/terminus-2>
- 10 Terminal-Bench Team. (2026). Terminal-Bench 2.1 leaderboard. <https://www.tbench.ai/leaderboard/terminal-bench/2.1>
- 11 Yao, Y., et al. (2026). Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows. <https://arxiv.org/abs/2605.27922>
- 12 Chen, S., et al. (2026). TUA-Bench: A Benchmark for General-Purpose Terminal-Use Agents. <https://arxiv.org/abs/2606.28480>
- 13 Mavali, S., et al. (2026). No More, No Less: Task Alignment in Terminal Agents. <https://arxiv.org/abs/2605.12233>
- 14 Anthropic. (2025). Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- 15 Anthropic. (2026). Harness design for long-running application development. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- 16 OpenAI. (2026). Codex: AI coding agents for software engineering. <https://openai.com/codex/>
- 17 University of Technology Sydney. (2026). 36127 Innovation Lab Capstone Project List - Spring 2026, Project 15 brief supplied by Dr William So, Synogize.
- 18 University of Technology Sydney. (2026). 36127 iLab Project Kickoff Slides - Spring 2026.
- 19 Anaissi, A. (2026). Capstone group formation and project preferences. Canvas announcement, 27 July 2026.